Chittagong University of Engineering and Technology



Lab Report

Experiment No. 02: A simulation of heart rate counter.

Course: Body Sensor Network sessional (BME 422)

Prepared by:

Name: Rudra Chakraborty

ID: 2011026

Submitted to:

Ayesha Binth Humayun,
Lecturer
Department of Biomedical Engineering,CUET

Submission Date: January 05, 2026

Objectives:

 System Simulation: To develop and validate a heartbeat monitoring system within the Proteus environment using an Arduino Uno and an HB1 sensor, focusing on real-time BPM calculation via pulse interval measurement.

 Algorithmic Implementation: To create a dependable signal processing framework that identifies pulse peaks through the Arduino's ADC, ensuring seamless data visualization on an LM016L LCD and Virtual Terminal.

Task 1: Design and Implementation of Threshold-Based Heart Rate Counter

The primary objective of this project was to develop a digital system capable of measuring heart rate using fixed-threshold peak detection techniques. The project was executed through the following stages:

System Architecture: An Arduino Uno R3 served as the main processing unit and was interfaced with a heartbeat sensor to acquire physiological signals.

Visual Output Interface: A 16×2 LM016L LCD was connected to the Arduino using digital pins 12, 11, 5, 4, 3, and 2 to display real-time BPM values. To improve the user interface, a custom heart symbol was designed using a dedicated byte array.

Signal Observation: The raw analog output waveform from the sensor's VOUT pin was monitored using a digital oscilloscope. Simultaneously, serial data logging was performed through a virtual terminal operating at a baud rate of 9600.

Peak Detection Algorithm: The Arduino continuously sampled analog input from pin A0. A fixed threshold value of 550 was set, and any signal exceeding this level was identified as a heartbeat peak.

Time-Based Analysis and BPM Computation: The program used the millis() function to measure the time intervals between five successive detected peaks. The heart rate was then calculated using the corresponding formula.

$$Heart_rate = \frac{N_{Peaks} - 1}{Time2 - Time1} * 60{,}000$$

Schematic Diagram:

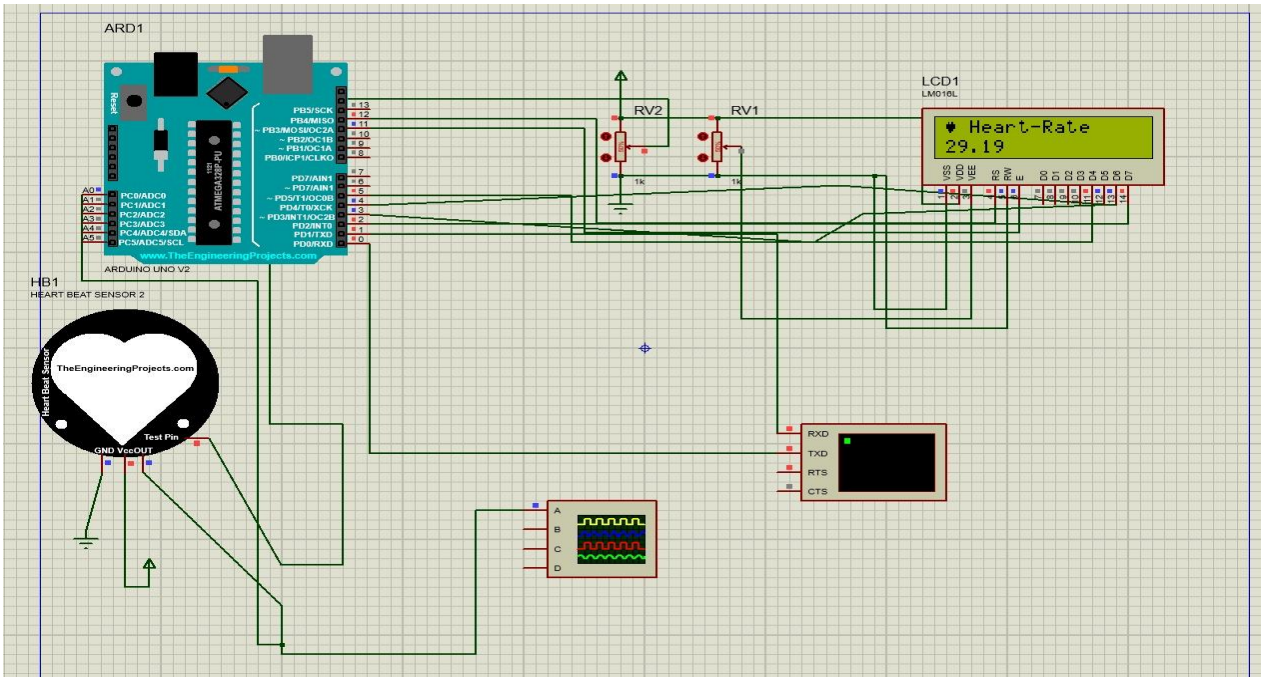


Figure: Low resistance input

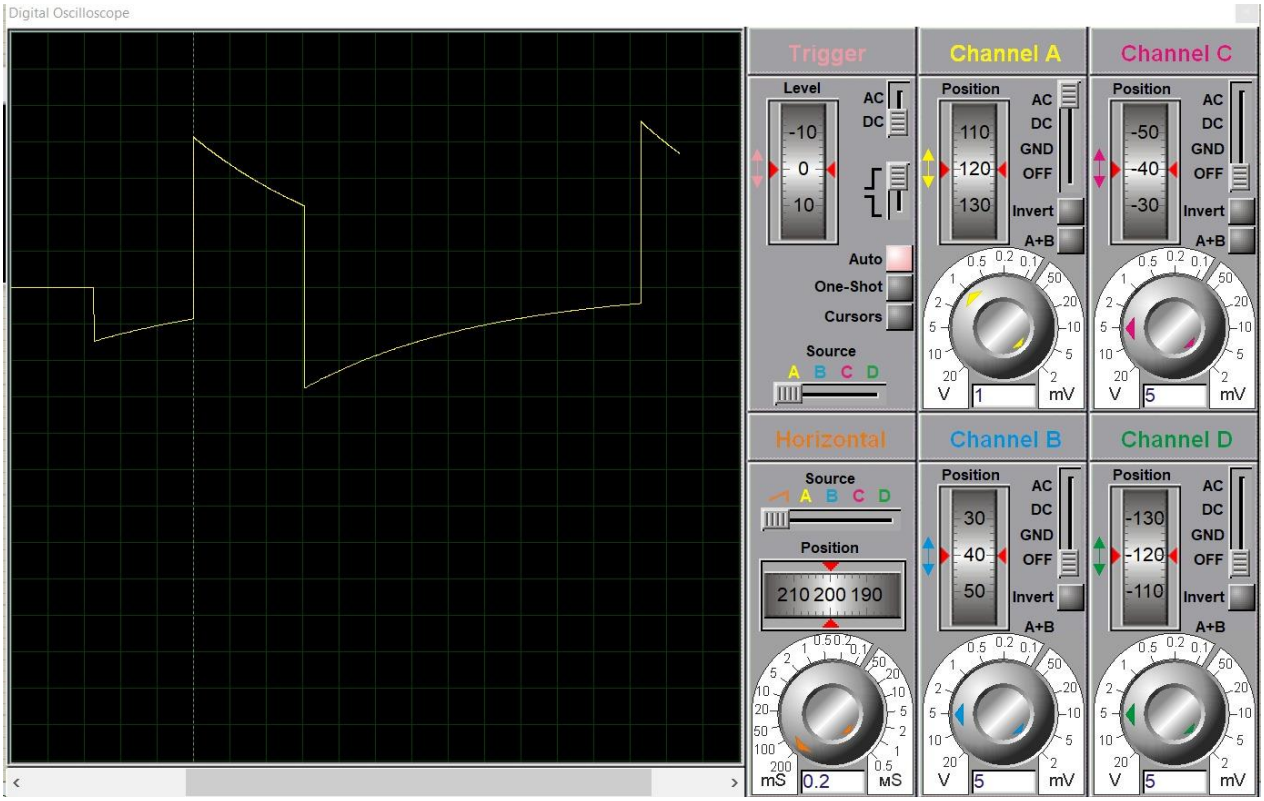


Figure: Peak detection

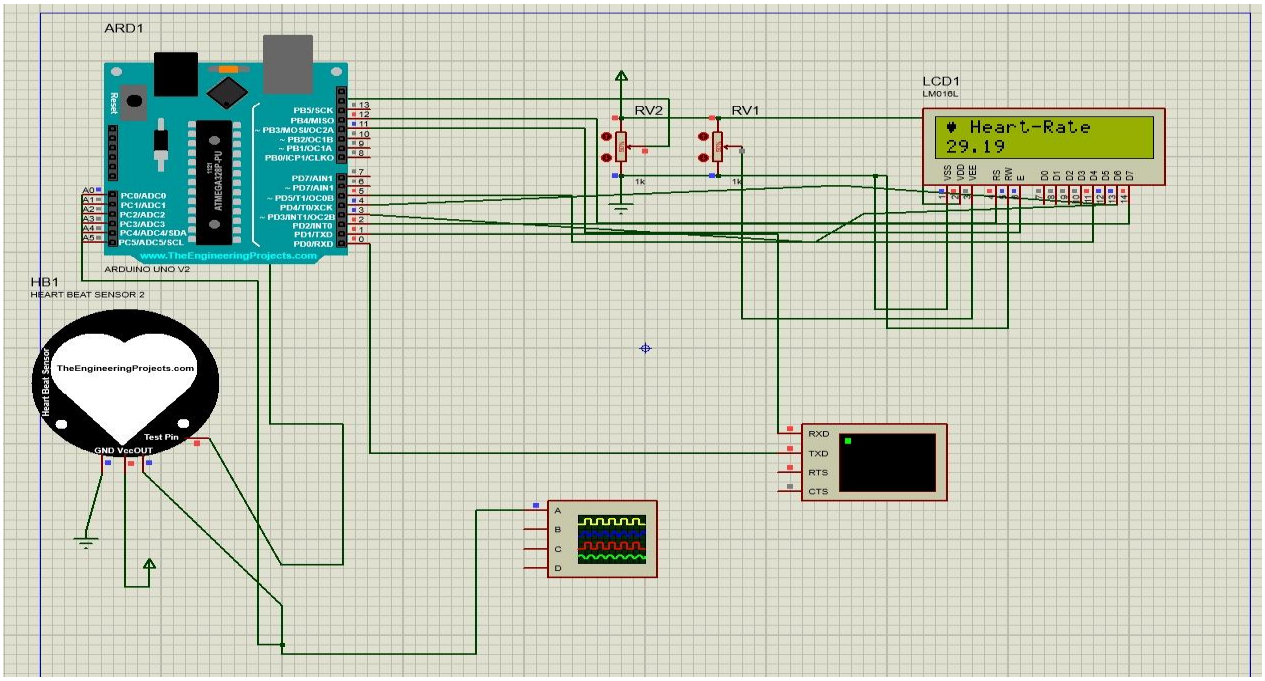


Figure: High Resistance Input

Code for Task-1:

```
// with the arduino pin number it is connected to
const int rs = 12, en = 11, d4 = 5, d5 = 4, d6 = 3, d7 =
2; LiquidCrystal lcd(rs, en, d4, d5, d6, d7); void setup()
{
Serial.begin(9600); // Set's up Serial Communication at certainspeed. // set up
the LCD's number of columns and rows:
lcd.createChar(1, heart);
lcd.begin(16, 2);
// Print a message to the
LCD. lcd.write(1);
lcd.print(" Heart-Rate");
time1 = 0; time2 = 0;
Npeaks = 0;
} void loop() {
while(Npeaks <
5){ delay(10);
// Read the PulseSensor's value.
Signal = analogRead(PulseSensorPurplePin);
// Assign this value to the "Signal" variable.
// Send the Signal value to Serial
Plotter. Serial.println(Signal);
if(Signal>Threshold && SignalPrev==0){
if(Npeaks == 0){time1=millis();}
Npeaks++;
}
SignalPrev = Signal>Threshold;
}
// store the time after the 5th pulse time2=millis();
//number of beats would be 1 less than number of pulses heartrate=(Npeaks-
1)/((time2-time1))*60000;
// set the cursor to column 0, line 1
// (note: line 1 is the second row, since counting begins with 0):
lcd.setCursor(0, 1);
lcd.print(heartrate);
// reset number of peaks to zero.
Npeaks = 0;
}
  time2 = millis();
  // Formula: (Total Beats / Total Time in ms) * 60,000 ms/min
heartrate = (Npeaks - 1) / (float(time2 - time1)) * 60000.0;
```

```
  lcd.setCursor(0, 1);   lcd.print("BPM: ");   lcd.print(heartrate);
  Npeaks = 0;
}
```

Task-2: Without using a threshold

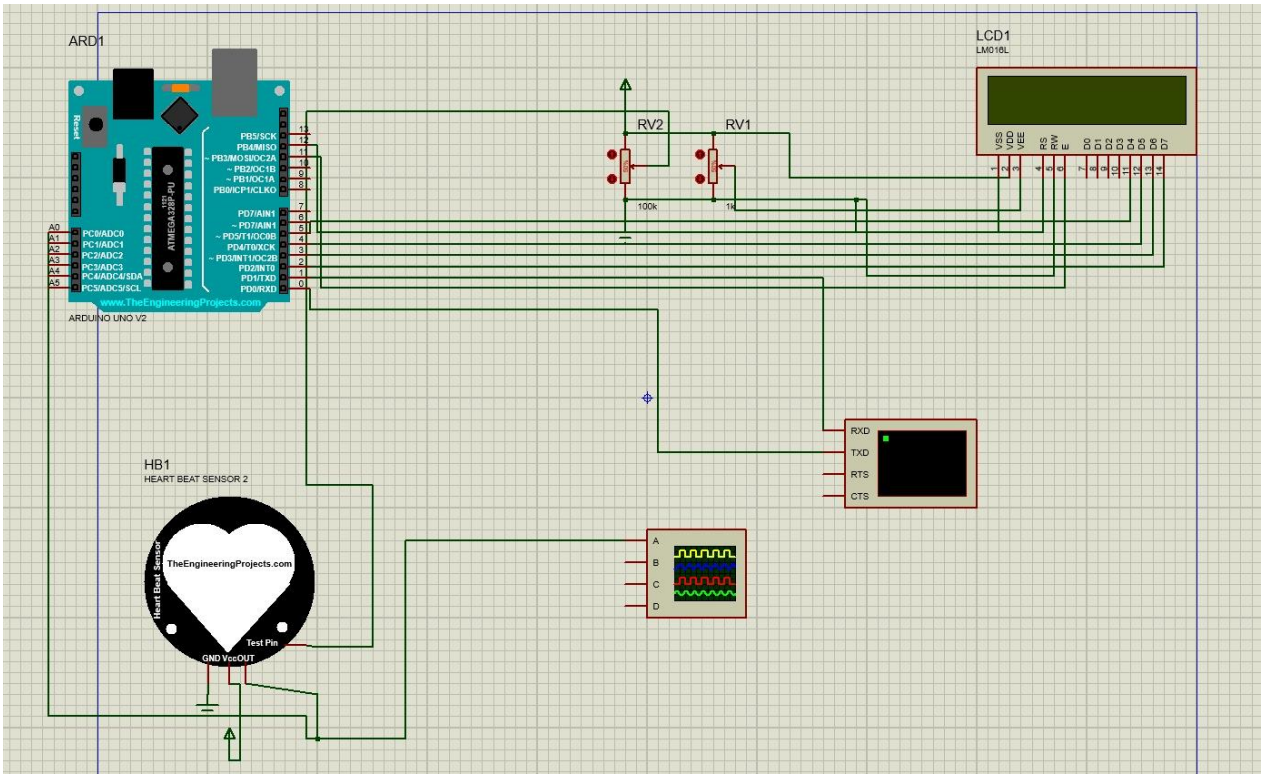


Figure: Schematic for task 2

Code for Task-2:

```
#include <LiquidCrystal.h>
 byte heart[8] = {0b00000, 0b01010, 0b11111, 0b11111, 0b11111, 0b01110,
0b00100,
0b00000};
const int rs = 12, en = 11, d4 = 5, d5 = 4, d6 = 3, d7 = 2;
LiquidCrystal lcd(rs, en, d4, d5, d6, d7);

int Signal, SignalPrev = 0; bool isRising = false;
float Npeaks = 0; float heartrate; unsigned long
time1 = 0, time2 = 0, lastPeakTime = 0;
```

```
void setup() {
Serial.begin(9600);
lcd.createChar(1, heart);
lcd.begin(16, 2);
lcd.write(1);
lcd.print(" Heart-Rate");
}  void loop() {
while(Npeaks < 5) {
delay(10);
    Signal = analogRead(0); // Reading from A0
    Serial.println(Signal);

    // --- SLOPE DETECTION LOGIC (NO THRESHOLD) --
-     if (Signal > SignalPrev) {       isRising =
true;
    }      else if (Signal < SignalPrev && isRising) {       unsigned
long currentMillis = millis(); // Using native millis(), no library

      // Debounce/Lockout to prevent noise
triggers       if (currentMillis - lastPeakTime >
250) {          if(Npeaks == 0) time1 =
currentMillis;         Npeaks++;
lastPeakTime = currentMillis;
      }       isRising
= false;      }
    SignalPrev = Signal;
  }
  time2 = millis();
  // Formula: (Total Beats / Total Time in ms) * 60,000 ms/min
heartrate = (Npeaks - 1) / (float(time2 - time1)) * 60000.0;

  lcd.setCursor(0, 1);
lcd.print("BPM: ");
lcd.print(heartrate);
  Npeaks = 0;
}
```

Feature	Task 1 Code (Threshold Method)	Task 2 Code (Slope Method)
Key Variable	int Threshold = 550	bool is rising
Detection Trigger	Signal > Threshold	Signal < SignalPrev && isRising
Signal Comparison	Compares the current signal to a fixed constant	Compares the current signal to the previous raw data sample.
Noise Handling	Relies on the signal dropping back below the threshold (550)	Uses a time-based "lockout" with millis() to prevent multiple triggers per peak.
Constraint Adherence	Simple logic using basic variables	Fulfills the requirement to detect peaks without a threshold and without timer libraries.

Discussion

This experiment focused on developing a heart rate monitoring system and comparing two detection techniques: Threshold-Based detection and Slope-Detection. In Task 1, heartbeat peaks were identified when the HB1 sensor's analog output crossed a fixed threshold of 550. While effective for stable signals, this method is limited due to its dependence on signal amplitude.

System timing accuracy was evaluated using the HG potentiometer (RV2) to simulate different heart rates. With a resistance of 1 kΩ, the system measured 29.19 BPM, while at 100 kΩ, the heart rate decreased to 24.14 BPM. These results confirm that the Arduino software accurately tracks temporal changes in the input signal.

Task 2 introduced a Slope-Detection algorithm that detects peaks by identifying the transition from rising to falling signal values, removing the need for a fixed threshold. This amplitude-independent approach provides improved reliability. Accurate timing was maintained using the millis() function without relying on external timer libraries.